DSA

INTRODUCTION

Data structure:

→ a named collection that can be used to store and organize data.

Algorithm:

→ a collection of steps to solve a problem

(the steps to a solution)

Primitive Data Structure are basic data structures provided by programming languages to represent single values such as integers, floating-point numbers, characters and booleans.

Abstract Data Structures are high-level data structures that are built using primitive data types and provide more complex and specialized operations. Some common examples of abstract data structures include arrays, linked lists, stacks, queues, trees and graphs.

ARRAY LISTS

→ a data structure used to store multiple elements.

→ is indexed, meaning that each element in the array has an index, a number that says where in the array the element is located. The programming languages use zero-based indexing for arrays.

→ stores elements in contiguous memory locations

→ we need to shift all elements that follow in order to make room for insertion, or close any gaps in the case of deletion

```
import java.util.ArrayList;
import java.util.

public class Main{
    public static void main(String[] args){
        ArrayList<String> array= new ArrayList<>();
        array.add("apple");
        array.add("banana");
        array.add("cherry");                    +size()
        System.out.println(array);
        System.out.println(array.get(1));
        array.set(1, "blueberry");
        array.remove("apple");
        for(String fruit: array)
            System.out.println(fruit);
    }
}
```

DYNAMIC ARRAYS

→ a static array has a fixed capacity (we cannot increase the capacity once the size of the elements reaches capacity)
→ an array with a resizeable capacity
→ if reaches capacity, it will create and instantiate a new array with an increased capacity.
→ usually the amount that we increase the capacity varies depending on the programming language
  - Java 1.5
  - Python 1.125
  - C++ 1.5
→ the elements of the new array have different memory addresses than the original array
→ easy to insert / delete at the end
→ wastes more memory than a linked list

```
public class DynamicArray{
    private int size;
    private int capacity=10;
    private Object[] array;

    public DynamicArray(){
        this.array = new Object [capacity];
    }

    public DynamicArray(int capacity)
        this.capacity= capacity;
        this.array = new Object [capacity];
    }
```

```
public int get_size(){
    return size;
}

public int get_capacity(){
    return capacity;
}

public Object get_array(){
    return array;
}

public void set_capacity(int capacity){
    this.capacity = capacity;
}

public void set_size(int size){
    this.size = size;
}

public void set_array(Object [] newArray){
    for(int i = 0
```

```
public void add(Object data){
    if(size >= capacity)
        grow();
    array[size++] = data;
}
```

```
pubeic void insert(imt imdex, Object dodo){
    if(size >= capacity)
        grow();
    for(imt i=size; i>imdex; i--){
        array[i]= array[i-1];
    array[imdex]= dodo;
    size++;
}
pubeic void deete(Object dodo){
    for(imt i=0; i< size; i++)
        if(array[i]== dodo){
            for(imt j=0; j<(size-i-1); j++)
                array[i+j]= array[i+j+1];
            array[--size]= nuee;
            if(size <= (imt)(capacity/3))
                shrink();
            break;
        }
}
pubeic imt search(Object dodo){
    for(imt i=0; i<size; i++)
        if(array[i]== dodo)
            return i;
    return -1;
}
```

```
private void grow(){
    int newCapacity = (int)(get_capacity() * 2);
    Object[] newArray = new Object[newCapacity];
    for(int i=0; i< get_size(); i++)
        newArray[i] = array[i];
    set_capacity(newCapacity);
    set_array(newArray);
}

private void shrink(){
    int newCapacity = (int)(get_capacity() / 2);
    Object[] newArray = new Object[newCapacity];
    for(int i=0; i< get_size(); i++)
        newArray[i] = array[i];
    set_capacity(newCapacity);
    set_array(newArray);
}

pubeic boolean isEmpty(){
    return get_size() == 0;
}
```

STACK

→ a LIFO data structure that can hold many elements
→ basic operations that we can do on a stack are:
- push(item)
- pop() → it returns the last item
- peek()
- isEmpty()
- size()

→ stacks can be implemented by using arrays or linked lists
→ stacks can be used to implement undo mechanisms, to revert to previous states, to create algorithms for depth-first search in graphs, or for backtracking.

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<String> myStack = new Stack<String>();
        myStack.push("John");
        myStack.push("Anna");
        System.out.println(myStack.peek());
        System.out.println(myStack.search("John")); //2
                                                    if it doesn't exist, it prints -1
```

QUEUE → first come first serve

→ a FIFO data structure that can hold many elements

→ basic operations that we can do on a queue are: enqueue → offer(item)
dequeue → poll()
peek()
isEmpty()
size()
↳ it doesn't throw an exception

→ queues can be implemented by using arrays or linked lists.

→ queues can be used to implement job scheduling for an office printer, order processing for e-tickets, or to create algorithms for breadth-first search in graphs.

```
import java.util.Queue;
import java.util.LinkedList;

public class Main{
    public static void main(String[] args){
        Queue<String> myQueue = new LinkedList<String>(); // we cannot instantiate the type Queue, because it is actually an interface, not a class.
        myQueue.offer("Kerem");
        myQueue.offer("Ahad");
        System.out.println(queue);
        System.out.println(queue.peek());
        myQueue.pole();
```

↳ or PriorityQueue

→ the Queue class will extend the Collection class, so the Queue class inherits everything that the Collection class has.

- `.isEmpty()` → it returns true or false
- `.size()`
- `.contains(item);` → it doesn't show the index

PRIORITY QUEUE

→ a FIFO data structure that can hold many elements
→ it serves elements with the highest priorities first before elements with lower priority

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Queue<Double> queue = new PriorityQueue<>();
        queue.offer(3.0);
        queue.offer(2.5);
        queue.offer(4.0);
        queue.offer(1.5);
        while (!queue.isEmpty()) {
            System.out.println(queue.poll());
        }
    }
}
```

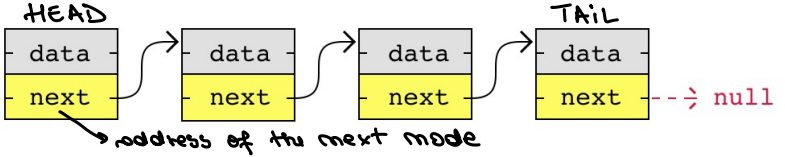
we pass between ( )
Collections.reverseOrder()

1.5
2.0
2.5
3.0
} if we want them reversed

! Strings are displayed in alphabetical order

LINKED LIST

→ a list where the nodes are linked together. Each node contains data and a pointer.
→ each node points to where in the memory the next node is placed.



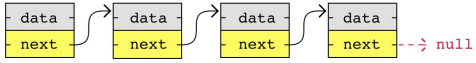
→ nodes are stored wherever there is free space in memory
→ nodes do not have to be stored contiguously right after each other like elements are stored in arrays.
→ when adding or removing nodes, the rest of the nodes in the list do not have to be shifted.
→ it doesn't have an index, so we cannot access a node directly
→ is used in many scenarios, like dynamic data storage, stack and queue implementation or graph representation (GPS navigation, music playlist)
→ searching: O(n) = linear time
→ insertion / deletion: O(1) = constant

Types of linked list:

→ single
→ double (requires more memory to store 2 addresses in each node)
→ circular

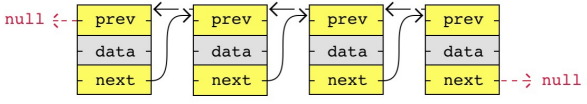
1. Singly linked list:

- it takes up less space in memory because each node has only one address to the next node



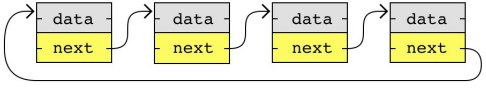
2. Doubly linked list:

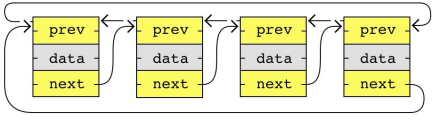
- has nodes with addresses to both the previous and the next node
- takes up more memory
- you are able to move both up and down in the list



3. Circular linked list:

- is like a singly or doubly linked list with the first node, the "head", and the last node, the "tail", connected
- in singly or doubly linked lists, we can find the start and end of a list by checking if the links are null. But for circular linked lists, more complex code is needed to explicitly check for start and end nodes.
- it cycles through continuously.



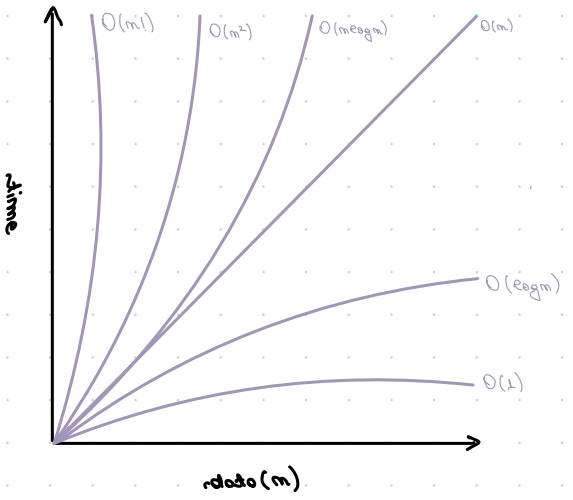


```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<String>();
        list.add("A");
        list.add("B");
        list.add("C");
        list.add("D");
        list.add("E");
        System.out.println(list); // [A, B, C, D, E]
        list.add(3, "X"); // [A, B, C, X, D, E]
        list.remove("E"); // [A, B, C, X, D]
        System.out.println(list.indexOf("D")); // 4
        System.out.println(list);
        System.out.println(list.peekFirst()); // A
        System.out.println(list.peekLast()); // D
        list.addFirst("a"); // [a, A, B, C, X, D]
        list.addLast("d"); // [a, A, B, C, X, D, d]
        list.removeFirst();
        list.removeLast();
    }
}
```

WE CAN TREAT IT AS A STACK OR AS A QUEUE

BIG O NOTATION



TIME COMPLEXITY:

O(1) = constant time

O(log n) = logarithmic time

O(n) = linear time

O(n log n) = quasilinear time

$O(n^2)$ = quadratic time

O(n!) = factorial time

LINEAR SEARCH

→ compares each value with the value it is looking for. If the value is found, the index is returned, and if it is not found -1 is returned.

```
private static int linearSearch(int[] array, int value){
    for(int i=0; i<array.length; i++)
        if(array[i]==value)
            return i;
    return -1;
}
```

BEST CASE SCENARIO: if the value we are looking for is the first value in the array.
(complexity: O(1))

WORST CASE SCENARIO: if the whole array is looked through without finding the target value
(complexity: O(n))

AVERAGE CASE SCENARIO: it depends on the values in the array
(complexity: O(n))

BINARY SEARCH

→ finds the target value in an already sorted array by checking the middle value. If the middle value is not the target value, binary search selects the left or right sub-array and continues the search until the target value is found.

```
private static void binarySearch(int[] array, int target){
        int low = 0;
        int high = array.length - 1;
        while(low <= high){
                int middle = low + (high - low)/2; //avoids overflow
                if(target == array[middle])
                        return middle;
                else{
                        if(target < array[middle])
                                high = middle - 1;
                        else
                                low = middle + 1;
                }
        }
        return -1;
}
```

```
private static void binarySearch(int[] array, int target, int low, int high){
    if(low > high)
        return -1;
    int middle= low+(high-low)/2; //avoids overflow
    if(target == array[middle])
        return middle;
    else {
        if(target < array[middle])
            return binarySearch(array, target, low, middle-1);
        else
            return binarySearch(array, target, middle-1, high);
    }
```

BEST CASE SCENARIO: if the first middle value is the same as the target value
(complexity: $O(1)$)

WORST CASE SCENARIO: if the search area must be cut in half over and over until the search area is just one value.
(complexity: $O(\log_2 n)$)

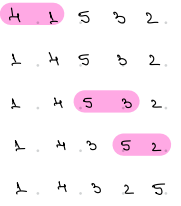
INTERPOLATION SEARCH

BUBBLE SORT

→ every value is compared to the next and swaps the values if the left value is larger than the right.

complexity: $O(n^2)$

```
private static void bubbleSort(int[] array){
    for(int i=0; i<array.length-1; i++)
        for(int j=0; j<array.length-1; j++)
            if(array[j]>array[j+1])
                swap(array[j], array[j+1]);
}
```



SELECTION SORT

→ finds the lowest value in the array and moves it to the front of the array, and does this over and over until the array is sorted.

Complexity: $O(n^2)$

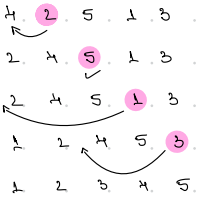
```
private static void bubbleSort(int[] array){
    for(int i=0; i<array.length-2 ;i++)
        for (int j=i+1; j<array.length-1; j++)
            if( array[i]> array[j])
                swap(array[i],array[j]);
}
```

INSERTION SORT

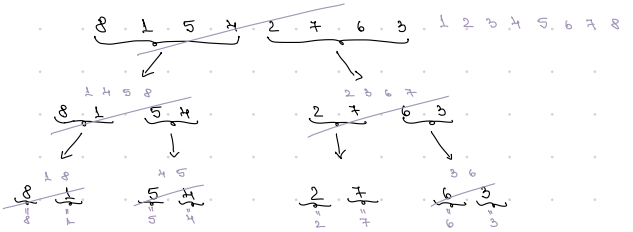
→ the first value is already in the correct position
→ the second value must be compared and moved past the first value
→ the third value must be compared and moved past two values
→ the fourth value must be compared and moved past three values
....

complexity: $O(n^2)$

```
private static void insertionSort(int[] array){
    for(int i=1; i<array.length; i++){
        int j=i;
        while(a[j]<a[j-1] && j>=1){
            swap(a[j], a[j-1]);
            j--;
        }
    }
}
```



MERGE SORT



→ breaks the array down into smaller and smaller pieces

→ becomes sorted when the sub-arrays are merged back together so that the lowest value comes first.

complexity: O(n log n)

```
public static void MergeSort(int[] a, int low, int high){
    if (low == high)
        return;
    else {
        int middle = low + (high - low)/2;
        MergeSort(a, low, middle);
        MergeSort(a, middle+1, high);
        merge(a, int low, int middle, int high);
    }
}
```

```
private static void Merge(int[] a, int low, int middle, int high){
    int[] temp= new int[high - low + 1];
    int i = low;
    int j = middle+1;
    int k = 0;
    while(i<= middle && j<= high){
        if(a[i]<= a[j])
            temp[k++]= a[i++];
        else
            temp[k++]= a[j++];
    }
    while(i<= middle)
        temp[k++]= a[i++];
    while(j<= high)
        temp[k++]= a[j++];
    System.arraycopy(temp, 0, a, low, temp.length);
}
```

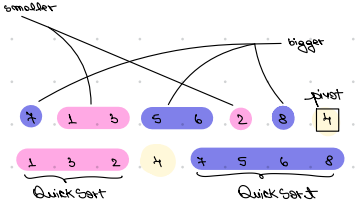
QUICK SORT

→ chooses a value as the 'pivot' element, and moves the other values so that higher values are on the right of the pivot element, and the lower values are on the left of the pivot element.

WORST CASE: if the array is already sorted. There is only one sub-array after each recursive call, and the new sub-arrays are only one element shorter than the previous array.

(complexity: $O(n^2)$)

AVERAGE CASE: it splits in half each time and the size of the sub-arrays decrease really fast ⟹ not so many recursive calls are needed.




```
public void int partition ( int[] a, int low, int high){
    int pivot = a[high];
    int current_index = low;
    for(int i = low; i < high; i++)
        if (a[i] <= pivot){
            swap(a[i], a[current_index]);
            current_index++;
        }
    swap (a[high], a[current_index]);
    return current_index;
}
```

```
public static void QuickSort(int[] a, int low, int high){
    if (low < high){
        int index = partition(a, low, high);
        QuickSort(a, low, index-1);
        QuickSort(a, index+1, high);
    }
}
```

HASH TABLES

→ is a data structure that stores unique keys to values
→ each key/value pair is known as an Entry
→ adding, look up and deleting data can be done really quickly, even for large amounts of data
→ we use the hash % capacity to calculate an index number
→ bucket = an indexed storage location for one or more entities
    ↳ can store multiple entries in case of collision (each entry knows where the next entry is located)
→ collision = hash function generates the same index for more than one key
    ↳ less collision = more efficiency
→ designed to be fast to work with

! Not ideal for small data sets, great with large data sets

BEST CASE SCENARIO: O(1) = there are no collisions
WORST CASE SCENARIO: O(n) = there are exclusively collisions

```
public class Main {
    public static void main(String[] args) {
        HashTable<Integer, String> table = new HashTable<>(10);
        table.put(100, "Spongebob");
        table.put(123, "Patrick");
        table.put(321, "Sandy");
        table.put(555, "Squidward");
        table.put(777, "Gary");
```

```
for(Integer key: tobee.keySet())
    System.out.printf(key.hashCode()%10 + "\t" + key + "\t" + tobee.get(key));
```

```
*  777  Crony
5  555  Squidword
0  123  Patrick
1  321  Sandy
0  100  Spongebob
```

```
tobee.remove(777);
```

```
// if we used strings, it would look like this.
HashMap<String, String> tobee = new HashMap<>(10);
tobee.put("100", "Spongebob");
tobee.put("123", "Patrick");
tobee.put("321", "Sandy");
tobee.put("555", "Squidword");

for(String key: tobee.keySet())
    System.out.printf(key.hashCode() + "\t" + key + "\t" + tobee.get(key));
```

```
52629  555  Squidword
48625  100  Spongebob
50610  321  Sandy
48690  123  Patrick
```

GRAPHS

→ are non-linear data structures that consists of nodes and edges

→ node = a point or an object in the Graph

→ edge = is used to connect two nodes with each other

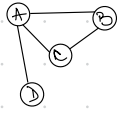
→ are non-linear because the data structure allows us to have different paths to get from one node to another, unlike with linear data structures like Arrays or Linked Lists.

PROPERTIES

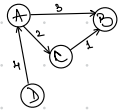
- **WEIGHTED:** a weighted graph is a graph where the edges have values. The weight value of an edge can represent things like distance, capacity, time, etc.
- **CONNECTED:** all the nodes are connected through edges. A graph that isn't connected, is a graph with isolated subgraphs or single isolated nodes. The direction of an edge can represent things like hierarchy or flow
- **DIRECTED** the edges between the node pairs have a direction.
- **CYCLIC** → DIRECTED CYCLIC: when you can follow a path along the directed edges that goes in circles.
  → UNDIRECTED CYCLIC: when you can come back to the same node you started at without using the same edge more than once.
- **LOOP**: an edge that begins and ends on the same vertex. A loop is a cycle that only consists of one edge.

Adjacency Matrix Graph Representation:

→ is a 2D array (matrix) where each cell on index (i, j) stores information about the edge from node i to node j.



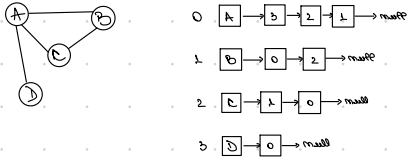
	A	B	C	D
A	0	1	1	1
B	1	0	1	0
C	1	1	0	0
D	1	0	0	0

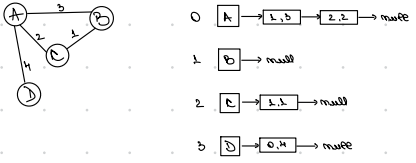


	A	B	C	D
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

Adjacency List Graph Representation:

→ has an array that contains all the nodes in the graph, and each node has a Linked List (or Array) with the node's edges.





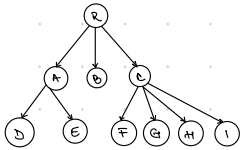
Depth First Search:

→ a search algorithm for traversing a tree or graph data structure

TREES

→ it's similar to Linked Lists in that each node contains data and can be linked to other nodes.

→ a single element can have multiple "next" elements, allowing the data structure to branch out in various directions.



It can be useful in many cases:

- Hierarchical Data: file systems, organizational models, etc.
- Databases: used for quick data retrieval.
- Routing Tables: used for routing data in network algorithms
- Sorting / Searching: used for sorting data and searching for data.
- Priority Queues: implemented using binary heaps.

Terminology and rules:

→ the first node in a tree is called the **root** node.

→ a link connecting one node to another is called an **edge**.

→ a **parent** node has links to its **child** nodes. Another word for a parent node is **internal** node.

→ a node can have zero, one or many child nodes.

→ a node can only have one parent node.

→ nodes without links to other child nodes are called **leaves**, or **leaf nodes**.

→ the **tree height** is the maximum number of edges from the root node to a leaf node.

→ the **weight of a node** is the maximum number of edges between the node and a leaf node.

→ the **tree size** is the number of nodes in the tree.

Types of Trees:

BINARY TREES: each node has up to two children, the left child node and the right child node.

BINARY SEARCH TREES: the left child node has a lower value and the right child node has a higher value.

AVL TREES: self-balances so that for every node, the difference in height between the left and right subtrees is at most one. This balance is maintained through rotations when nodes are inserted or deleted.